Operating Systems

Project Checkpoint 5

112070002 張博安

Table of Contents

---

1 Part 1

1.1 Compilation

```
boanz@BENSONCH /cygdrive/d/OS/ppc5_1
$ make clean
rm *.hex *.ihx *.lnk *.lst *.map *.mem *.rel *.rst *.sym *.asm *.lk
rm: cannot remove '*.hex': No such file or directory
rm: cannot remove '*.ihx': No such file or directory
rm: cannot remove '*.lnk': No such file or directory
rm: cannot remove '*.lst': No such file or directory
rm: cannot remove '*.map': No such file or directory
rm: cannot remove '*.mem': No such file or directory
rm: cannot remove '*.rel': No such file or directory
rm: cannot remove '*.rst': No such file or directory
rm: cannot remove '*.sym': No such file or directory
rm: cannot remove '*.asm': No such file or directory
rm: cannot remove '*.lk': No such file or directory
make: *** [Makefile:18: clean] Error 1

boanz@BENSONCH /cygdrive/d/OS/ppc5_1
$ make
sdcc -c  --model-small testlcd.c
sdcc -c  --model-small preemptive.c
preemptive.c:149: warning 85: in function ThreadCreate unreferenced function arg
ument : 'fp'
preemptive.c:190: warning 158: overflow in implicit constant conversion
sdcc -c  --model-small lcdlib.c
lcdlib.c:76: warning 85: in function delay unreferenced function argument : 'n'
sdcc -c  --model-small buttonlib.c
sdcc -c  --model-small keylib.c
sdcc  -o testlcd.hex testlcd.rel preemptive.rel lcdlib.rel buttonlib.rel keylib.
rel
```

Figure 1: `make clean` and `make` for Part 1

1.2 Address List

```
      Value  Global                              Global Defined In Module
      -----  -----------------------------      ------------------------
C:   00000081  _Producer1                         testlcd
C:   000000D5  _Producer2                         testlcd
C:   00000129  _Consumer                          testlcd
C:   0000016B  _main                              testlcd
C:   0000018C  __sdcc_gsinit_startup              testlcd
C:   00000190  __mcs51_genRAMCLEAR                testlcd
C:   00000191  __mcs51_genXINIT                   testlcd
C:   00000192  __mcs51_genXRAMCLEAR               testlcd
C:   00000193  _timer0_ISR                        testlcd
C:   00000199  _Bootstrap                         preemptive
C:   000001C2  _myTimer0Handler                   preemptive
C:   0000024B  _ThreadCreate                      preemptive
C:   000002BE  _ThreadYield                       preemptive
C:   00000312  _ThreadExit                        preemptive
C:   0000036F  _LCD_ready                         lcdlib
C:   00000373  _LCD_Init                          lcdlib
C:   00000386  _LCD_IRWrite                       lcdlib
C:   000003A8  _LCD_functionSet                   lcdlib
C:   000003D2  _LCD_write_char                    lcdlib
C:   000003FC  _LCD_write_string                  lcdlib
C:   00000431  _delay                             lcdlib
C:   00000435  _AnyButtonPressed                  buttonlib
C:   00000447  _ButtonToChar                      buttonlib
C:   000004D3  _Init_Keypad                       keylib
C:   000004D9  _AnyKeyPressed                     keylib
C:   000004E6  _KeyToChar                         keylib
C:   00000562  __gptrget                          _gptrget
```

Figure 2: Function address list for Part 1

1.3 Producer 1 (Button Bank)

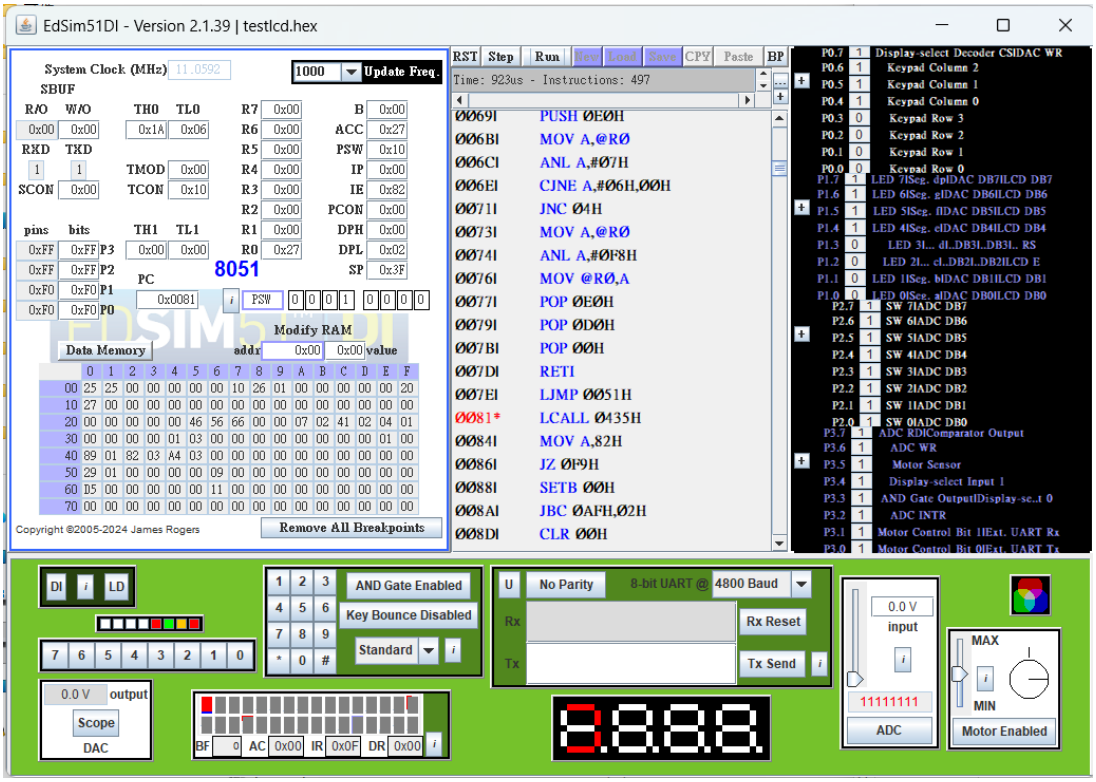

Figure 3: Screenshot for the first `Producer1` call

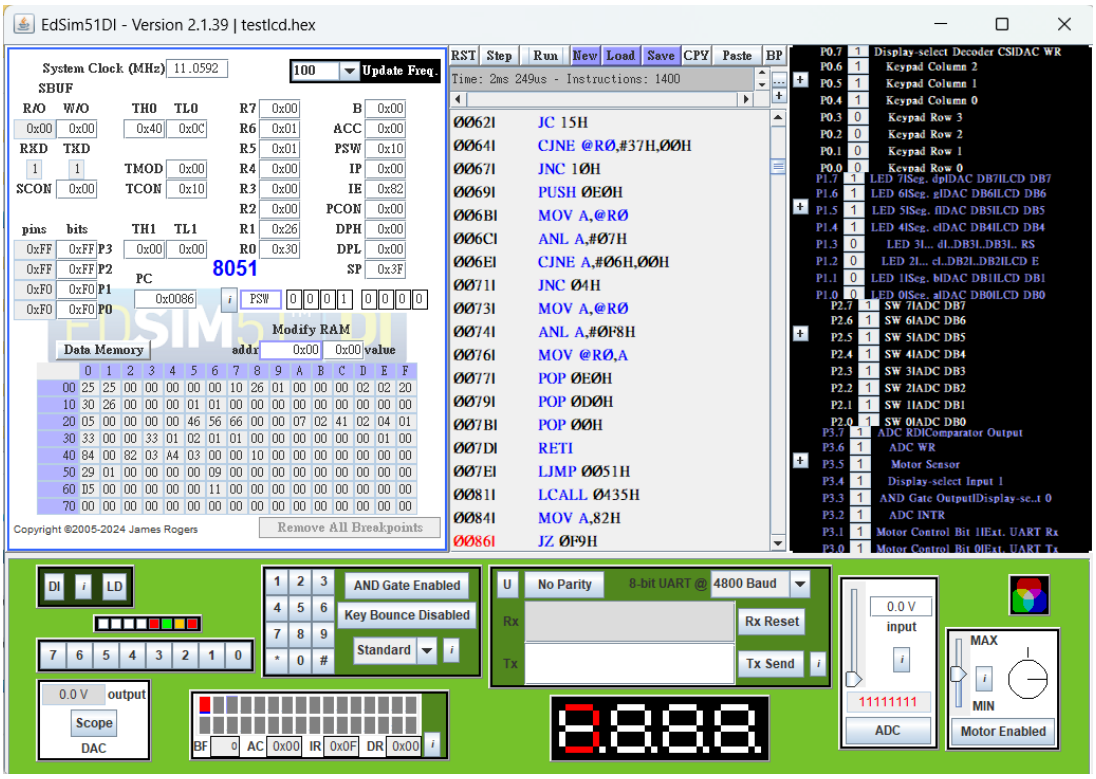

Figure 4: Screenshot after pressing button 3 in `Producer1`

The `Producer1` function reads from the button bank (port `P2`). Its code starts at address `0x0081`. It busy-waits on `AnyButtonPressed` until a button is pressed, then reads the button value via `ButtonToChar` inside a critical section to prevent a race condition with the timer interrupt. The ASCII value is enqueued into the shared ring buffer at `0x30--0x32` after acquiring the `empty` and `mutex` semaphores. After signalling `mutex` and `full`, it waits for the button to be released before the next read. In the example above, button 3 was pressed, so `0x33` (ASCII `'3'`) was written to `buffer[0]` at address `0x30`.

1.4 Producer 2 (Keypad)

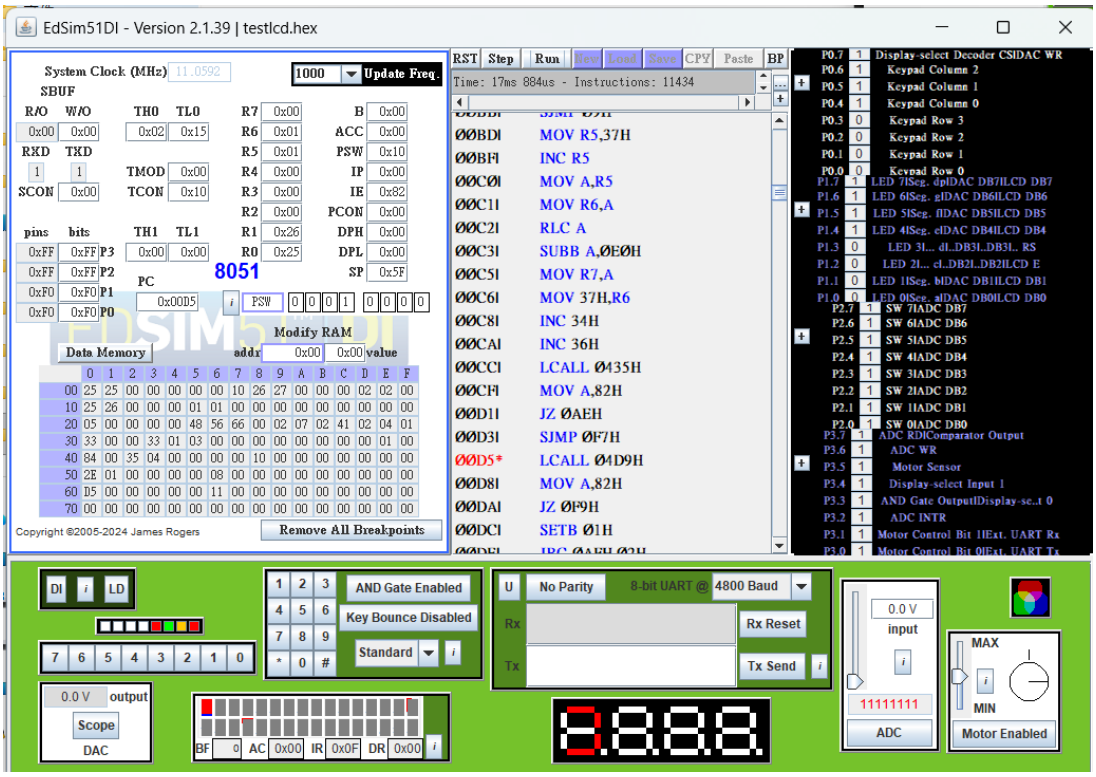

Figure 5: Screenshot for the first `Producer2` call

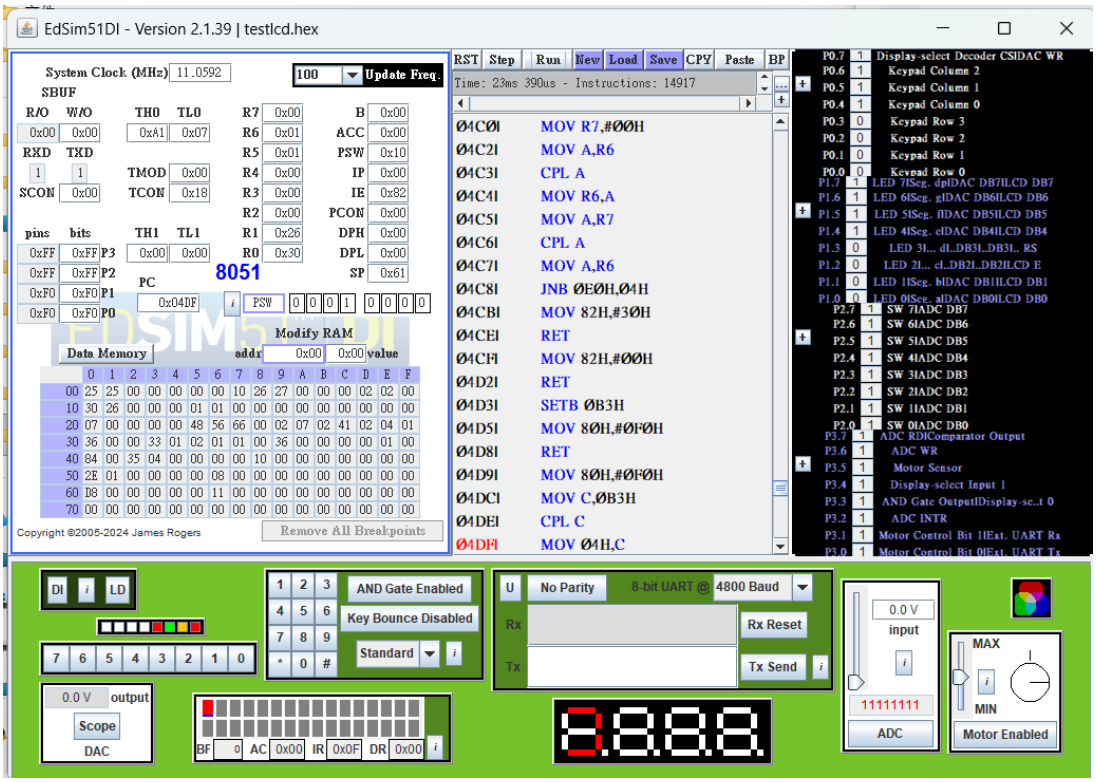

Figure 6: Screenshot after pressing keypad key 6 in `Producer2`

The `Producer2` function reads from the 3×4 matrix keypad. Its code starts at address `0x00D5`. The structure is identical to `Producer1`: the keypad is scanned row by row via `KeyToChar`, with `AnyKeyPressed` using the AND-gate output on `P3.3` for a quick any-key detection. The ASCII value is enqueued into the shared buffer under semaphore protection, and the function waits for the key to be released before proceeding. In the example above, keypad key 6 was pressed, so `0x36` (ASCII `'6'`) was written to `buffer[0]` at address `0x30`.

1.5 Consumer (LCD)

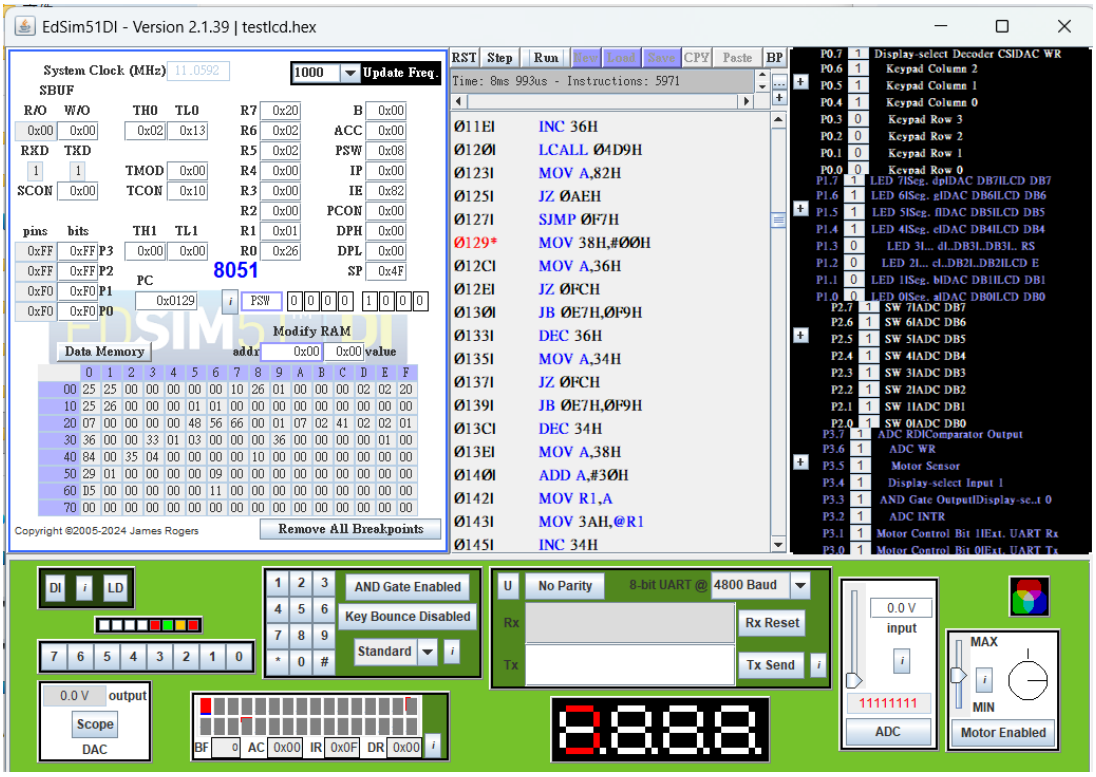

Figure 7: Screenshot for the first `Consumer` call

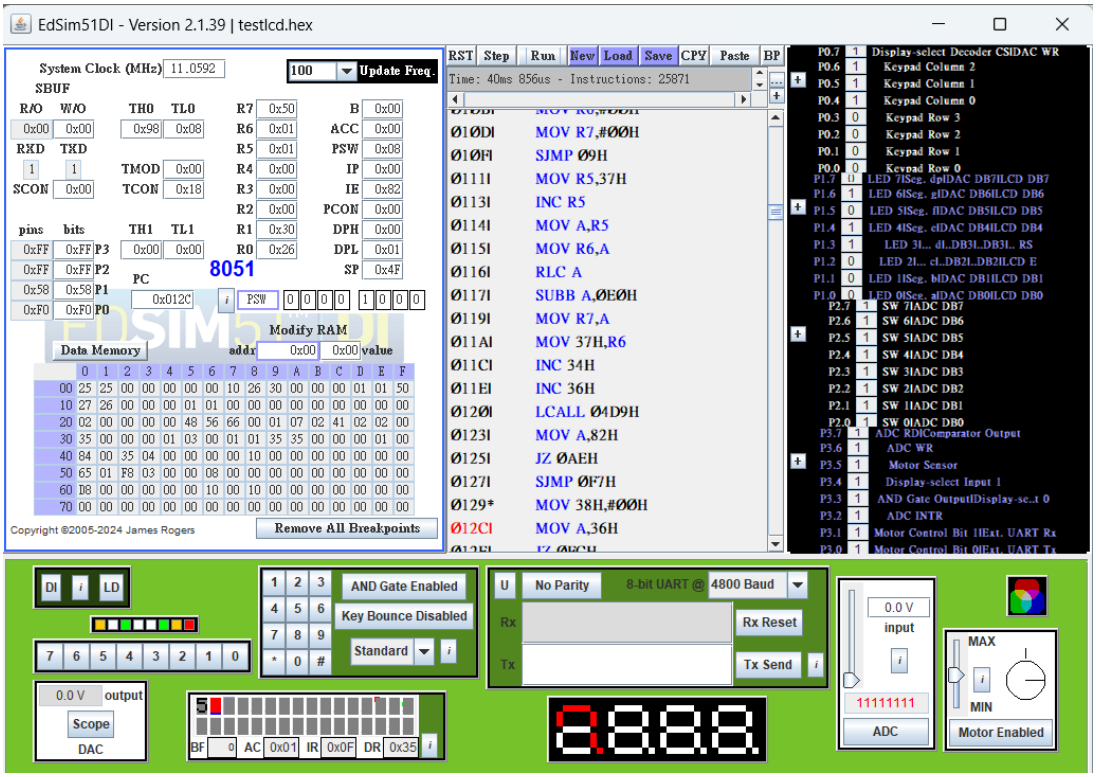

Figure 8: Screenshot of the LCD displaying character `'5'`

The `Consumer` function's code starts at address `0x0129`. It dequeues one character at a time from the shared ring buffer under semaphore protection, then writes it to the LCD via `LCD_write_char`. After each write it busy-waits on `LCD_ready` before issuing the next character, because the HD44780 controller requires a settling delay between consecutive writes. In the example above, keypad key 5 was pressed, so `0x35` (ASCII `'5'`) was enqueued and subsequently displayed on the LCD. The LCD Data Register (`DR`) confirms the value `0x35` was correctly transmitted to the display.

2 Part 2

2.1 Compilation

```
boanz@BENSONCH /cygdrive/d/OS/ppc5_2
$ make clean
rm *.hex *.ihx *.lnk *.lst *.map *.mem *.rel *.rst *.sym *.asm *.lk
rm: cannot remove '*.ihx': No such file or directory
rm: cannot remove '*.lnk': No such file or directory
make: *** [Makefile:16: clean] Error 1

boanz@BENSONCH /cygdrive/d/OS/ppc5_2
$ make
sdcc -c  --model-small dino.c
sdcc -c  --model-small preemptive.c
preemptive.c:240: warning 85: in function ThreadCreate unreferenced function argument : 'fp'
preemptive.c:333: warning 158: overflow in implicit constant conversion
sdcc -c  --model-small lcdlib.c
lcdlib.c:80: warning 85: in function delay unreferenced function argument : 'n'
sdcc -c  --model-small keylib.c
sdcc  -o dino.hex dino.rel preemptive.rel lcdlib.rel keylib.rel
```

Figure 9: `make clean` and `make` for Part 2

2.2 Address List

```
      Value  Global                          Global Defined In Module
      -----  --------------------------------  ------------------------
C:    00000081  _render_task                    dino
C:    000001B6  _keypad_ctrl                    dino
C:    00000237  _game_ctrl                      dino
C:    00000319  _main                           dino
C:    0000039B  __sdcc_gsinit_startup           dino
C:    0000039F  __mcs51_genRAMCLEAR             dino
C:    000003A0  __mcs51_genXINIT                dino
C:    000003A1  __mcs51_genXRAMCLEAR            dino
C:    000003A2  _timer0_ISR                     dino
C:    000003A8  _Bootstrap                      preemptive
C:    000003C6  _myTimer0Handler                preemptive
C:    00000422  _ThreadCreate                   preemptive
C:    0000049C  _ThreadYield                    preemptive
C:    000004F7  _ThreadExit                     preemptive
C:    00000566  _ThreadReset                    preemptive
C:    0000056A  _LCD_ready                      lcdlib
C:    0000056E  _LCD_Init                       lcdlib
C:    000005CF  _LCD_IRWrite                    lcdlib
C:    000005F1  _LCD_functionSet                lcdlib
C:    0000061B  _LCD_write_char                 lcdlib
C:    00000645  _LCD_write_string               lcdlib
C:    0000067A  _delay                          lcdlib
C:    0000067E  _LCD_set_symbol                 lcdlib
C:    0000078F  _Init_Keypad                    keylib
C:    00000795  _AnyKeyPressed                  keylib
C:    000007A2  _KeyToChar                      keylib
C:    0000081E  __gptrget                       _gptrget
```

Figure 10: Function address list for Part 2

2.3 Main Function

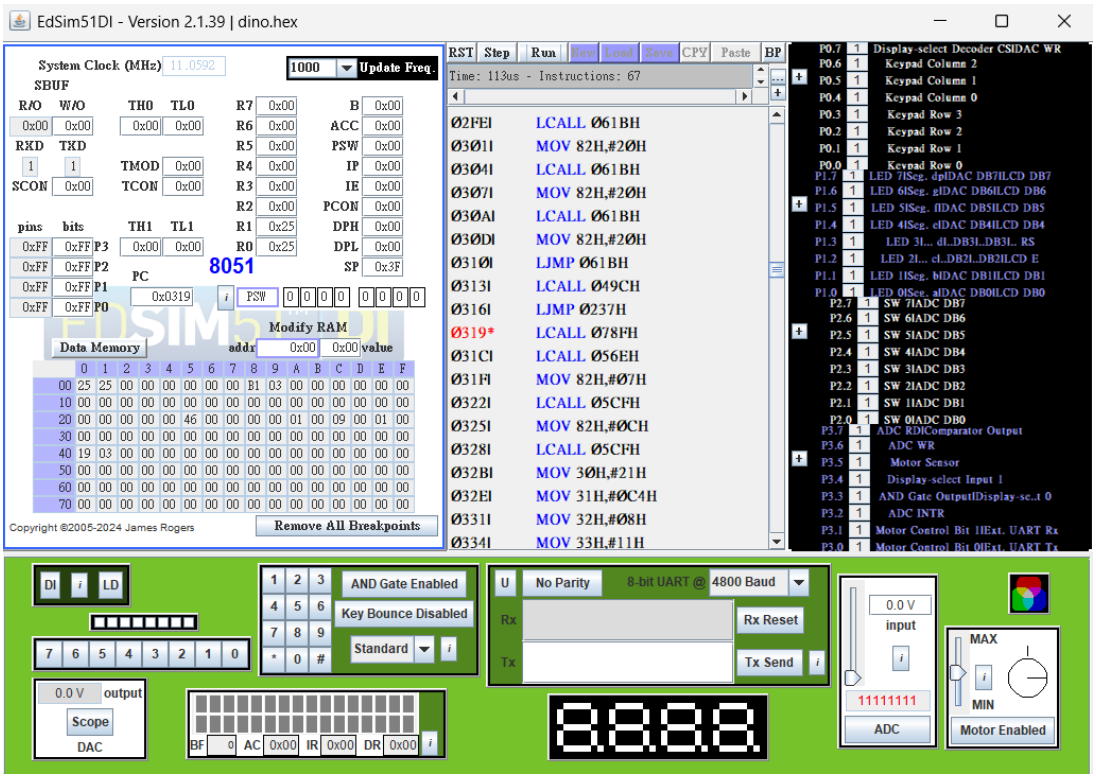

Figure 11: Screenshot for the first `main` call

The `main` function's code starts at address `0x0319`. It first initialises the keypad and LCD, then enters an infinite loop that supports multiple rounds of play. At the start of each round, the cactus bitmaps (`row_top` and `row_bot`), score, difficulty level, and player position are all reset. The user is then prompted to enter a digit (1–9) followed by `#` to set the difficulty level; pressing `#` without a prior digit is ignored. After the difficulty is confirmed, `render_task` and `keypad_ctrl` are spawned as Thread 1 and Thread 2. Timer 0 is configured in mode 0 with `TH0 = (level << 4)`, so a higher difficulty produces a faster overflow and more frequent context switches, making the cacti scroll faster. `game_ctrl` is then called directly in Thread 0. When `game_ctrl` returns after a collision, interrupts are disabled and `ThreadReset` kills Threads 1 and 2, leaving only Thread 0 alive to start the next round.

2.4 Render Task Thread

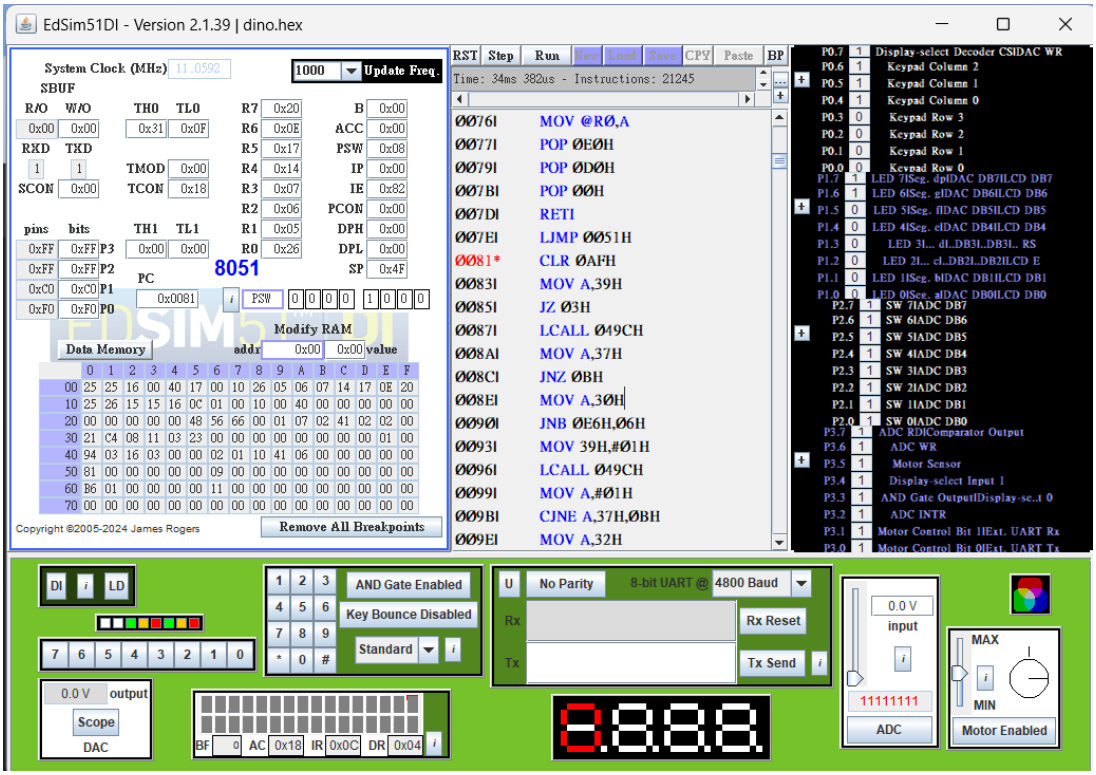

Figure 12: Screenshot for the first `render_task` call

The `render_task` function's code starts at address `0x0081`. It runs as an infinite loop and is responsible for updating the LCD every frame. At the start of each frame, interrupts are disabled (`EA = 0`) and the function checks whether `gameover` is set; if so, it immediately yields without drawing. It then checks for a collision at column 1 (bit `0x40` of `row_top[0]` or `row_bot[0]`), since that is the column the player occupies after the next shift. If a collision is detected, `gameover` is set to 1 and the thread yields.

Otherwise, both cactus bitmaps are shifted left by one bit. The most significant bit of each bitmap pair is saved as a carry flag before the shift; if a cactus exits the left edge, it means the player successfully dodged it, so the score is incremented and the cactus is reinserted at the rightmost column (bit 0 of `row_top[1]` or `row_bot[1]`).

After shifting, both rows are redrawn on the LCD using custom character `'\2'` for cacti and a space for empty columns. Finally, the player sprite (`'\1'`) is written to column 0 of `dino_row`, and `ThreadYield` is called.

The cactus map is stored as four `unsigned char` variables (`row_top[0]`, `row_top[1]`, `row_bot[0]`, `row_bot[1]`), using one bit per column position. This requires only 4 bytes instead of 32, which is critical given the 8051's limited internal RAM.

2.5 Keypad Control Thread

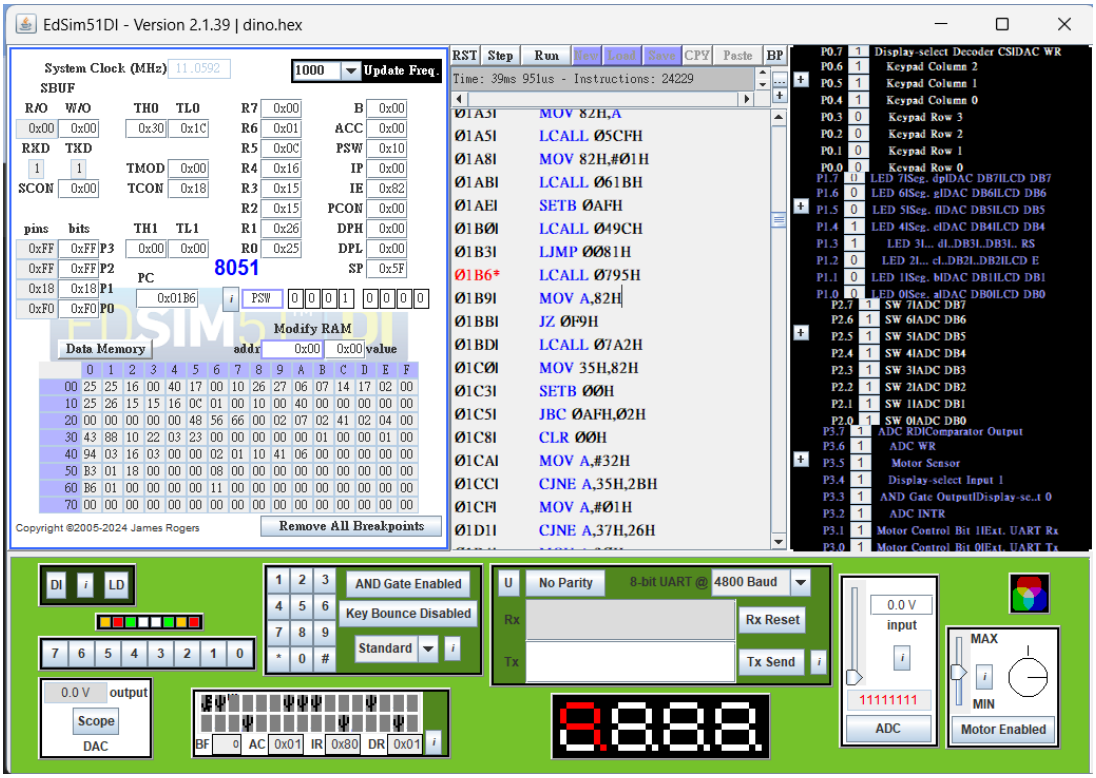

Figure 13: Screenshot for the first `keypad_ctrl` call

The `keypad_ctrl` function's code starts at address `0x01B6`. It busy-waits on `AnyKeyPressed` until a key is held down, then reads the key with `KeyToChar`. The movement logic runs inside a `__critical` block to prevent the timer interrupt from firing mid-update and causing a race condition where two player sprites could appear simultaneously on the LCD.

Pressing `'2'` moves the player from row 1 to row 0 (upward). Pressing `'8'` moves the player from row 0 to row 1 (downward). Before moving, the function checks bit `0x80` of the destination row's bitmap (column 0); if a cactus is already there, `gameover` is set and the thread yields immediately instead of moving. After a valid move, the old position is overwritten with a space and the new position is drawn with the player sprite. The function then busy-waits for the key to be released before reading the next input.

2.6 Game Control Thread

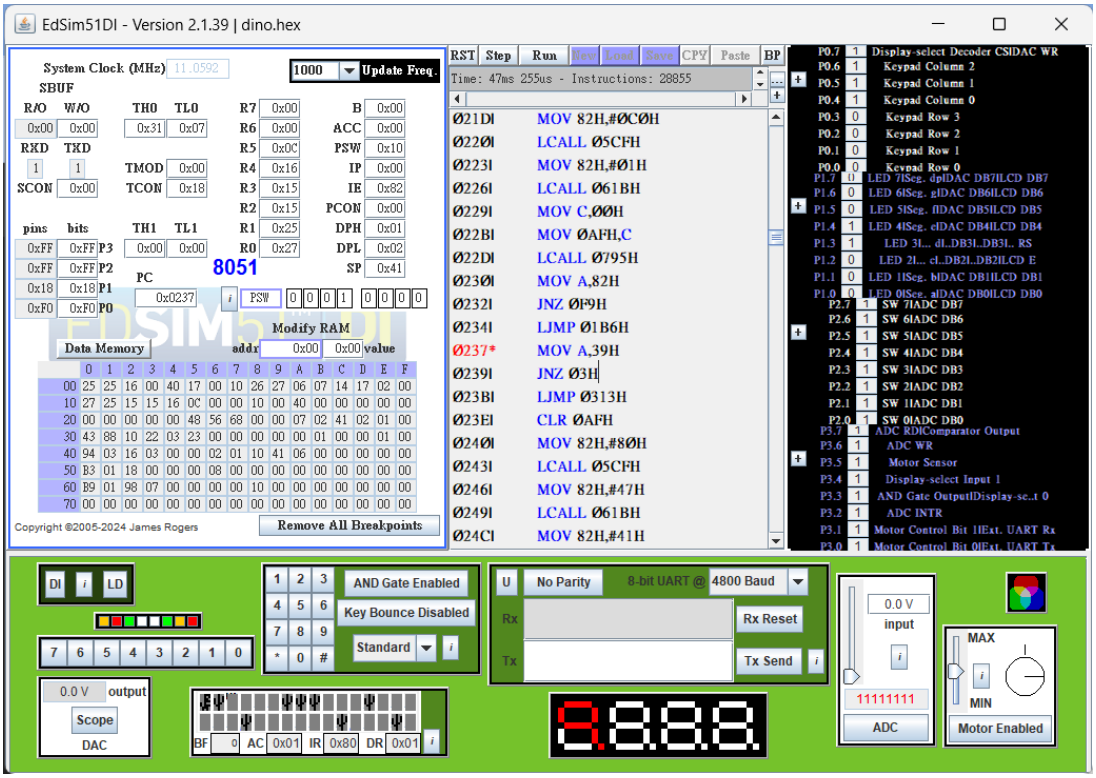

Figure 14: Screenshot for the first `game_ctrl` call

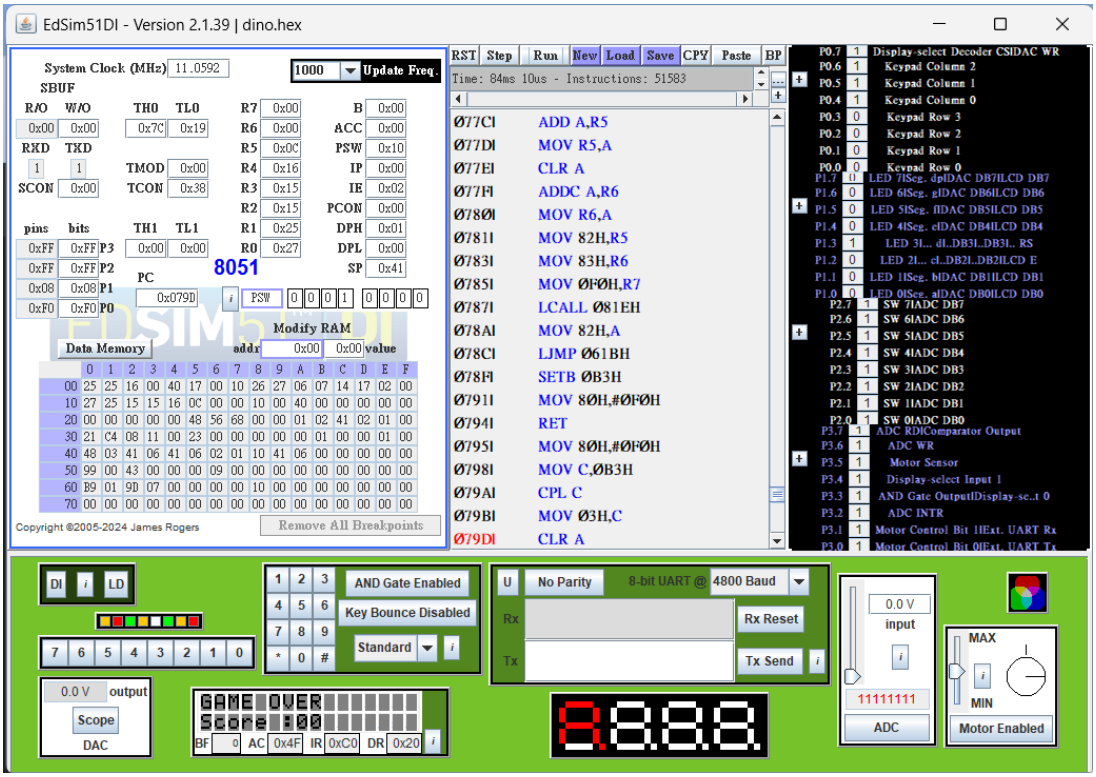

Figure 15: Screenshot of the LCD after the dinosaur collides with a cactus

The game_ctrl function's code starts at address 0x0237. It shares Thread 0 with main and runs as an infinite loop that simply calls ThreadYield until gameover becomes 1. Once a collision is detected, interrupts are disabled and the LCD is overwritten with GAME OVER on the first row and the score in decimal on the second row. The score is displayed as two digits using integer division and modulo. After writing the game-over screen, the function returns to main, which resets the thread mask and restarts the game loop.

Q1: What data type do you use for the map?

The map is stored as four unsigned char variables: row_top[0], row_top[1], row_bot[0], and row_bot[1]. Each pair represents one LCD row as a 16-bit bitmap where each bit corresponds to one column. This requires only 4 bytes instead of 32, which is essential given the 8051's limited internal RAM.

Q2: How do you generate a new cactus?

A predefined starting map is used. Each frame render_task performs a circular left shift: a cactus that exits the left edge is reinserted at the rightmost column. The initial values are chosen to avoid vertical or diagonal walls, so the pattern remains valid indefinitely.

2.7 Race Condition

The primary race condition occurs in keypad_ctrl. If the timer interrupt fires between reading AnyKeyPressed and updating dino_row and the LCD, render_task could redraw the old player position while keypad_ctrl is writing the new one, resulting in two player sprites on screen simultaneously. This is resolved by wrapping the movement update in a __critical block, which disables interrupts for the duration of the LCD write and position update.